

ARCH TECHNOLOGIES

Sharpening your hidden skills for a brighter future

MONTH 2 INTERNSHIP REPORT

Domain: Generative AI

Task 3: Implement RAG With Unsloth's Dynamic 4-bit Quantization

Task 4: Build A Speech-to-Reasoning Pipeline With Whisper & Quantized LLM

Submitted by: Zaka Saleem

Internship Domain: Generative AI

Email Address: zakasaleem35@gmail.com

Phone Number: +92 325 7380731

Introduction

Generative AI is a branch of artificial intelligence focused on creating new content such as text, images, audio, and code by learning patterns from existing data. This report documents the completion of the Month 2 tasks assigned under the Generative AI track at Arch Technologies, covering two projects: a Retrieval-Augmented Generation (RAG) pipeline built on an Unsloth dynamic 4-bit quantized model, and a speech-to-reasoning pipeline combining Whisper transcription with a quantized reasoning model.

Task 3: Implement RAG With Unsloth's Dynamic 4-bit Quantization

Objective

Build a Retrieval-Augmented Generation (RAG) pipeline using an Unsloth dynamic 4-bit quantized model. Index a set of domain-specific documents, build retrieval logic to fetch relevant chunks for a given query, and integrate this retrieval step with the quantized LLM so that responses are grounded in the retrieved context rather than generated purely from the model's own training.

Tools and Technologies Used

- Google Colab : T4 GPU runtime
- Unsloth : for loading and running the dynamic 4-bit quantized model
- Base model: unsloth/Llama-3.2-3B-Instruct-unsloth-bnb-4bit (dynamic quantization variant)
- Sentence-Transformers (all-MiniLM-L6-v2) : for generating text embeddings
- FAISS : for fast similarity search over document embeddings

Implementation Approach

A dynamic 4-bit quantized Llama 3.2 (3B) model was loaded using Unsloth's FastLanguageModel. Unlike standard uniform quantization, this dynamic variant selectively keeps higher precision for parameters that are more sensitive to quantization error, while compressing the rest more aggressively, this was visible in the loaded model's structure, where some layers used full-precision Linear layers instead of Linear4bit. A small knowledge base of eight domain-specific text passages covering Generative AI concepts was embedded using a Sentence-Transformers model and indexed with FAISS for fast similarity search. A retrieval function was built to fetch the most relevant passages for a given query, and a RAG function combined this retrieval step with the quantized LLM: retrieved passages were inserted into the prompt as context, and the model was instructed to answer only based on that context. The pipeline was tested with two sample queries to confirm both correct retrieval and grounded, context-based answer generation.

Code Implementation

```
from unsloth import FastLanguageModel
import torch

max_seq_length = 2048

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "unsloth/Llama-3.2-3B-Instruct-unsloth-bnb-4bit",
    max_seq_length = max_seq_length,
```

```

        dtype = None,
        load_in_4bit = True,
    )

FastLanguageModel.for_inference(model)

documents = [
    "Generative AI refers to models that can create new content such as text, images, audio, and code by learning patterns from existing data.",
    "Large Language Models (LLMs) are trained on massive text datasets and use transformer architectures to predict and generate human-like text.",
    "Retrieval-Augmented Generation (RAG) combines a retrieval system with a language model, allowing the model to fetch relevant information before generating a response, improving factual accuracy.",
    "Quantization reduces the precision of a model's weights (e.g., from 16-bit to 4-bit) to save memory and speed up inference, with minimal loss in output quality.",
    "LoRA (Low-Rank Adaptation) is a parameter-efficient fine-tuning technique that trains small adapter layers instead of updating all of a model's parameters.",
    "Unsloth is a library that makes fine-tuning and running large language models significantly faster and more memory-efficient, especially on consumer GPUs.",
    "Dynamic 4-bit quantization selectively preserves higher precision for critical model parameters while aggressively compressing less important ones, improving accuracy compared to uniform quantization.",
    "Vector embeddings represent text as numerical vectors, allowing semantic similarity between pieces of text to be measured using distance metrics like cosine similarity.",
]

from sentence_transformers import SentenceTransformer
import faiss
import numpy as np

embedder = SentenceTransformer("all-MiniLM-L6-v2")

doc_embeddings = embedder.encode(documents)
doc_embeddings = np.array(doc_embeddings).astype("float32")

dimension = doc_embeddings.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(doc_embeddings)

print(f"Indexed {index.ntotal} documents.")

def retrieve(query, top_k=2):
    query_embedding = embedder.encode([query]).astype("float32")
    distances, indices = index.search(query_embedding, top_k)
    retrieved_docs = [documents[i] for i in indices[0]]
    return retrieved_docs

def rag_query(user_query, top_k=2):
    retrieved_docs = retrieve(user_query, top_k=top_k)
    context = "\n".join(retrieved_docs)

    prompt = f"""Answer the question based only on the following context. If the context doesn't contain the answer, say you don't know.

Context:
{context}

```

```

Question: {user_query}

Answer: ""

    inputs = tokenizer([prompt], return_tensors="pt").to("cuda")
    outputs = model.generate(**inputs, max_new_tokens=200, use_cache=True)
    response = tokenizer.batch_decode(outputs, skip_special_tokens=True)[0]

    answer = response.split("Answer: ")[-1].strip()
    return answer, retrieved_docs

question = "What is quantization and why is it useful?"
answer, sources = rag_query(question)
print("Question:", question)
print("Retrieved Context:", sources)
print("Answer:", answer)

question2 = "What does RAG stand for and how does it work?"
answer2, sources2 = rag_query(question2)
print("Question:", question2)
print("Retrieved Context:", sources2)
print("Answer:", answer2)

```

Code Explanation

- `FastLanguageModel.from_pretrained()` loads the dynamic 4-bit quantized Llama 3.2 (3B) model, the 'unsloth-bnb-4bit' variant, as opposed to a uniformly quantized 'bnb-4bit' model, meaning some critical layers are kept at higher precision.
- The documents list acts as a small domain-specific knowledge base that the RAG pipeline retrieves from.
- `SentenceTransformer('all-MiniLM-L6-v2')` converts each document into a numerical embedding vector, and FAISS's `IndexFlatL2` builds a searchable index over these vectors.
- `retrieve()` embeds the user's query the same way and searches the FAISS index for the most similar documents (nearest neighbors in embedding space).
- `rag_query()` combines retrieval and generation: it retrieves relevant context, inserts it into a prompt template that instructs the model to answer only from that context, and generates a response using the quantized LLM.
- The two test queries at the end demonstrate the full pipeline end-to-end, confirming that retrieval correctly surfaces relevant passages and that the model's answers are grounded in that retrieved context rather than fabricated.

Results

The dynamic 4-bit quantized model loaded successfully on a T4 GPU, and its printed structure confirmed selective precision, most linear layers were quantized (Linear4bit) while at least one layer retained full-precision (Linear), consistent with dynamic quantization's design goal of preserving accuracy on sensitive parameters. All eight documents were successfully indexed. For the query "What is quantization and why is it useful?", the pipeline correctly retrieved the two most relevant passages (on quantization and dynamic quantization) and generated an accurate, grounded answer. For the query "What does RAG stand for and how does it work?", the pipeline again retrieved the correct primary passage and produced a correct, concise answer, confirming the RAG pipeline works reliably end-to-end.

Working Demonstration

```
... Unloth: Will patch your computer to enable 2x faster finefinetuning.  
🐍 Unloth Zoo will now patch everything to make training faster!  
==(====)= Unloth 2026.9.4: Fast Llama patching. Transformers: 5.5.0.  
    \ \ /| Tesla T4. Num GPUs = 1. Max memory: 14.563 GB. Platform: Linux.  
O^O/ \_/ \ Torch: 2.11.0+cu128. CUDA: 7.5. CUDA Toolkit: 12.8. Triton: 3.6.0  
    \ / Bfloat16 = FALSE. FA [Xformers = 0.0.35, FA2 = False]  
    "-_--_" Free license: http://github.com/unsothai/unsoth  
Unloth: Fast downloading is enabled - ignore downloading bars which are red colored!  
Loading weights: 100% ██████████ 254/254 [00:01<00:00, 1.45s/it]  
  
Unloth: Will load unsoth/Llama-3.2-3B-Instruct-unsoth-bnb-4bit as a legacy tokenizer.  
LlamaForCausalLM(  
  (model): LlamaModel(  
    (embed_tokens): Embedding(128256, 3072, padding_idx=128004)  
    (layers): ModuleList(  
      (0): LlamaDecoderLayer(  
        (self_attn): LlamaAttention(  
          (q_proj): Linear4bit(in_features=3072, out_features=3072, bias=False)  
          (k_proj): Linear4bit(in_features=3072, out_features=1024, bias=False)  
          (v_proj): Linear4bit(in_features=3072, out_features=1024, bias=False)  
          (o_proj): Linear4bit(in_features=3072, out_features=3072, bias=False)  
          (rotary_emb): LlamaRotaryEmbedding()  
        )  
      )  
    )  
  )  
)  
)
```

Figure 1: Dynamic 4-bit quantized model (unsloth-bnb-4bit variant) loaded successfully on the T4 GPU, showing the model's layer structure.

```
print(f"Indexed {index.ntotal} documents.")
```

modules.json: 100% 349/349 [00:00<00:00, 25.0kB/s]

config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 8.49kB/s]

README.md: 100% 10.5k/10.5k [00:00<00:00, 1.03MB/s]

sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 5.26kB/s]

config.json: 100% 612/612 [00:00<00:00, 26.5kB/s]

model.safetensors: reconstructing file: 100% 90.9MB / 90.9MB, 8.79MB/s

model.safetensors: downloading bytes: 85.0MB, 7.99MB/s

Loading weights: 100% 103/103 [00:00<00:00, 2332.84it/s]

BertModel LOAD REPORT for: sentence-transformers/all-MiniLM-L6-v2

Key	Status
embeddings.position_ids	UNEXPECTED

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.

tokenizer_config.json: 100% 350/350 [00:00<00:00, 15.1kB/s]

vocab.txt: 100% 232k/232k [00:00<00:00, 4.80MB/s]

tokenizer.json: 100% 466k/466k [00:00<00:00, 8.18MB/s]

Figure 2: Document embeddings generated and indexed successfully ("Indexed 8 documents.").

```
▶ question = "What is quantization and why is it useful?"
  answer, sources = rag_query(question)

  print("Question:", question)
  print("\nRetrieved Context:")
  for doc in sources:
    print("-", doc)
  print("\nAnswer:", answer)

... Both `max_new_tokens` (=200) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence. Please ref
Question: What is quantization and why is it useful?

Retrieved Context:
- Quantization reduces the precision of a model's weights (e.g., from 16-bit to 4-bit) to save memory and speed up inference, w
- Dynamic 4-bit quantization selectively preserves higher precision for critical model parameters while aggressively compressing

Answer: Quantization reduces the precision of a model's weights (e.g., from 16-bit to 4-bit) to save memory and speed up inference
```

Figure 3: RAG pipeline test : query about quantization, with retrieved context and generated answer.

```
▶ question2 = "What does RAG stand for and how does it work?"
  answer2, sources2 = rag_query(question2)

  print("Question:", question2)
  print("\nRetrieved Context:")
  for doc in sources2:
    print("-", doc)
  print("\nAnswer:", answer2)

... Both `max_new_tokens` (=200) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence. Please ref
Question: What does RAG stand for and how does it work?

Retrieved Context:
- Retrieval-Augmented Generation (RAG) combines a retrieval system with a language model, allowing the model to fetch relevant
- LoRA (Low-Rank Adaptation) is a parameter-efficient fine-tuning technique that trains small adapter layers instead of updating

Answer: RAG stands for Retrieval-Augmented Generation, and it combines a retrieval system with a language model to fetch relevant

Activate Windows
```

Figure 4: RAG pipeline test: query about RAG itself, with retrieved context and generated answer.

Task 4: Build A Speech-to-Reasoning Pipeline With Whisper & Quantized LLM

Objective

Build an end-to-end pipeline that takes a spoken audio query, transcribes it to text using OpenAI's Whisper, and passes the transcribed text to a quantized reasoning model (Unsloth dynamic 4-bit) to produce a clear, step-by-step reasoned answer.

Tools and Technologies Used

- Google Colab : T4 GPU runtime
- OpenAI Whisper ("base" model) : for speech-to-text transcription
- gTTS (Google Text-to-Speech) : to generate a sample spoken audio query for testing the pipeline
- Unsloth : for loading and running the dynamic 4-bit quantized reasoning model
- Base model : unsloth/Llama-3.2-3B-Instruct-unsloth-bnb-4bit (dynamic quantization variant)

Implementation Approach

A sample spoken query was generated using gTTS to simulate a real audio input, since no physical microphone recording was available in the Colab environment, the generated audio asked, "What is 15 percent of 240, and is that more than 30?" OpenAI's Whisper ("base" model) was then used to transcribe this audio file into text. The transcribed text was passed into a prompt template instructing the model to reason step-by-step, and this prompt was sent to the same dynamic 4-bit quantized Llama 3.2 (3B) model used in Task 3, loaded fresh via Unsloth's FastLanguageModel. The model's generated response was parsed and printed alongside the original transcribed question, demonstrating a complete voice-in, reasoned-answer-out pipeline.

Code Implementation

```
%%capture
!pip install unsloth
!pip install --upgrade --no-cache-dir "git+https://github.com/unslothai/unsloth.git"
!pip install -U openai-whisper
!pip install gTTS

import whisper

whisper_model = whisper.load_model("base")

from gtts import gTTS

text_to_speak = "What is 15 percent of 240, and is that more than 30?"

tts = gTTS(text=text_to_speak, lang='en')
tts.save("/content/sample_audio.mp3")

print("Audio file created at /content/sample_audio.mp3")

audio_path = "/content/sample_audio.mp3"
```

```

result = whisper_model.transcribe(audio_path)
transcribed_text = result["text"]

print("Transcribed Text:", transcribed_text)

from unsloth import FastLanguageModel
import torch

max_seq_length = 2048

reasoning_model, reasoning_tokenizer = FastLanguageModel.from_pretrained(
    model_name = "unsloth/Llama-3.2-3B-Instruct-unsloth-bnb-4bit",
    max_seq_length = max_seq_length,
    dtype = None,
    load_in_4bit = True,
)

FastLanguageModel.for_inference(reasoning_model)

prompt = f"""Answer the following question with clear step-by-step reasoning.

Question: {transcribed_text}

Answer: """

inputs = reasoning_tokenizer([prompt], return_tensors="pt").to("cuda")
outputs = reasoning_model.generate(**inputs, max_new_tokens=250, use_cache=True)
response = reasoning_tokenizer.batch_decode(outputs, skip_special_tokens=True)[0]

final_answer = response.split("Answer: ")[-1].strip()

print("Transcribed Question:", transcribed_text)
print("\nModel's Reasoning/Answer:", final_answer)

```

Code Explanation

- `whisper.load_model('base')` loads a lightweight Whisper model, which is fast enough to run on a T4 GPU while still giving accurate transcriptions.
- `gTTS` generates a spoken-audio version of a sample question and saves it as an `.mp3` file, standing in for a real microphone recording so the full pipeline can be tested end-to-end.
- `whisper_model.transcribe()` converts the audio file into text (`transcribed_text`), which is the bridge between the speech input and the reasoning model.
- `FastLanguageModel.from_pretrained()` loads the same dynamic 4-bit quantized Llama 3.2 (3B) model used for the RAG task, again in 4-bit precision for fast, memory-efficient inference on a free-tier GPU.
- The prompt template explicitly asks for 'clear step-by-step reasoning', which encourages the model to break the problem down rather than jumping straight to an answer.
- `reasoning_model.generate()` produces the response, which is then parsed to extract just the answer portion and printed alongside the original transcribed question.

Results

Whisper correctly transcribed the generated audio into the text "What is 15% of 240 and is that more than 30?", with no errors in the numbers or wording. The dynamic 4-bit quantized reasoning model loaded successfully, and its printed structure again confirmed selective precision, most layers used Linear4bit while at least one layer retained full-precision Linear, consistent with Task 3's findings. When given the transcribed question, the model produced a clear, correctly ordered, step-by-step solution: it converted 15% to a decimal (0.15), calculated 15% of 240 as 36, compared 36 to 30, and correctly concluded that the answer is yes, since 36 is greater than 30. This confirms the full speech-to-reasoning pipeline works correctly end-to-end, from spoken audio to a transcribed question to a reasoned, accurate answer.

Working Demonstration

```
result = whisper_model.transcribe(audio_path)
transcribed_text = result["text"]

print("Transcribed Text:", transcribed_text)
```

... Transcribed Text: What is 15% of 240 and is that more than 30?

Figure 5: Whisper transcription output — the spoken query correctly converted to text.

```
FastLanguageModel.for_inference(reasoning_model)
```

```
... Unsloth: Will patch your computer to enable 2x faster free finetuning.  
Unsloth Zoo will now patch everything to make training faster!  
==(=====)== Unsloth 2026.9.11: Fast Llama patching. Transformers: 5.5.0.  
\\    /| Tesla T4. Num GPUs = 1. Max memory: 14.563 GB. Platform: Linux.  
O^O/ \\_/ \ Torch: 2.11.0+cu128. CUDA: 7.5. CUDA Toolkit: 12.8. Triton: 3.6.0  
\\      / Bfloat16 = FALSE. FA [Xformers = 0.0.35. FA2 = False]  
"-_____" Free license: http://github.com/unslothai/unsloth  
Unsloth: Fast downloading is enabled - ignore downloading bars which are red colored!  
Loading weights: 100% ██████████ 254/254 [00:02<00:00, 2.09s/it]  
  
Unsloth: Will load unsloth/Llama-3.2-3B-Instruct-unsloth-bnb-4bit as a legacy tokenizer.  
LlamaForCausalLM(  
  (model): LlamaModel(  
    (embed_tokens): Embedding(128256, 3072, padding_idx=128004)  
    (layers): ModuleList(  
      (0): LlamaDecoderLayer(  
        (self_attn): LlamaAttention(  
          (q_proj): Linear4bit(in_features=3072, out_features=3072, bias=False)  
          (k_proj): Linear4bit(in_features=3072, out_features=1024, bias=False)  
          (v_proj): Linear4bit(in_features=3072, out_features=1024, bias=False)  
          (o_proj): Linear4bit(in_features=3072, out_features=3072, bias=False)  
          (rotary_emb): LlamaRotaryEmbedding()
```

Figure 6: Dynamic 4-bit quantized reasoning model loaded successfully on the T4 GPU.

```

    )
    (mlp): LlamaMLP(
      (gate_proj): Linear4bit(in_features=3072, out_features=8192, bias=False)
      (up_proj): Linear4bit(in_features=3072, out_features=8192, bias=False)
      (down_proj): Linear4bit(in_features=8192, out_features=3072, bias=False)
      (act_fn): SiLUActivation()
    )
    (input_layernorm): LlamaRMSNorm((3072,), eps=1e-05)
    (post_attention_layernorm): LlamaRMSNorm((3072,), eps=1e-05)
  )
  (1): LlamaDecoderLayer(
    (self_attn): LlamaAttention(
      (q_proj): Linear4bit(in_features=3072, out_features=3072, bias=False)
      (k_proj): Linear4bit(in_features=3072, out_features=1024, bias=False)
      (v_proj): Linear4bit(in_features=3072, out_features=1024, bias=False)
      (o_proj): Linear4bit(in_features=3072, out_features=3072, bias=False)
      (rotary_emb): LlamaRotaryEmbedding()
    )
    (mlp): LlamaMLP(
      (gate_proj): Linear(in_features=3072, out_features=8192, bias=False)
      (up_proj): Linear(in_features=3072, out_features=8192, bias=False)
      (down_proj): Linear(in_features=8192, out_features=3072, bias=False)
      (act_fn): SiLUActivation()
    )
    (input_layernorm): LlamaRMSNorm((3072,), eps=1e-05)
    (post_attention_layernorm): LlamaRMSNorm((3072,), eps=1e-05)
  )
  (2-27): 26 x LlamaDecoderLayer(
    (self_attn): LlamaAttention(
      (q_proj): Linear4bit(in_features=3072, out_features=3072, bias=False)
      (k_proj): Linear4bit(in_features=3072, out_features=1024, bias=False)
      (v_proj): Linear4bit(in_features=3072, out_features=1024, bias=False)
      (o_proj): Linear4bit(in_features=3072, out_features=3072, bias=False)
      (rotary_emb): LlamaRotaryEmbedding()
    )
    (mlp): LlamaMLP(
      (gate_proj): Linear4bit(in_features=3072, out_features=8192, bias=False)
      (up_proj): Linear4bit(in_features=3072, out_features=8192, bias=False)
      (down_proj): Linear4bit(in_features=8192, out_features=3072, bias=False)
      (act_fn): SiLUActivation()
    )
    (input_layernorm): LlamaRMSNorm((3072,), eps=1e-05)
    (post_attention_layernorm): LlamaRMSNorm((3072,), eps=1e-05)
  )
  )

```

Figure 7: Model layer structure showing dynamic quantization : Layer 1's MLP projections remain full-precision (Linear) while other layers use Linear4bit.

```

print("Transcribed Question:", transcribed_text)
print("\nModel's Reasoning/Answer:", final_answer)

... Both `max_new_tokens` (=250) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer to the
Transcribed Question: What is 15% of 240 and is that more than 30?

Model's Reasoning/Answer: To solve this problem, we will follow these steps:

## Step 1: Convert the percentage to a decimal
To find 15% of 240, we first need to convert 15% to a decimal. To do this, we divide by 100. So, 15% = 15/100 = 0.15.

## Step 2: Calculate 15% of 240
Now, we multiply 0.15 by 240 to find 15% of 240. This calculation is 0.15 * 240 = 36.

## Step 3: Compare the result to 30
Now that we know 15% of 240 is 36, we compare this result to 30 to determine if 36 is more than 30.

## Step 4: Conclusion
Since 36 is indeed greater than 30, the answer is yes.

The final answer is: $\boxed{\text{yes}}$

```

Figure 8: Final pipeline output : transcribed question and the model's step-by-step reasoned answer.

Conclusion

Both Month 2 tasks under the Generative AI track were completed successfully. Task 3 involved building a working Retrieval-Augmented Generation pipeline on top of an Unsloth dynamic 4-bit quantized model, combining document embeddings, FAISS-based similarity search, and grounded LLM generation to produce factually accurate, context-based answers. Task 4 involved building a complete speech-to-reasoning pipeline, combining OpenAI's Whisper for transcription with the same class of dynamic 4-bit quantized model to produce clear, step-by-step reasoned answers from spoken queries. Together, these tasks provided hands-on experience with retrieval-augmented systems, multi-modal pipelines that bridge speech and text, and the practical benefits of dynamic quantization for running capable language models efficiently on limited hardware.